



UNIVERSIDAD TECNOLÓGICA DE PANAMÁ
FACULTAD DE INGENIERÍA DE SISTEMAS COMPUTACIONALES
LICENCIATURA EN DESARROLLO Y GESTIÓN DE SOFTWARE

Laboratorio N° 3

Desarrollo de Software VII

Expresión regulares (Regex) y gestión de errores en PHP

PRESENTADO POR:

JORGE JIMÉNEZ	4-826-874
BRIANT ARANGO	4-825-620
JONATHÁN MARTÍNEZ	4-824-2482
DAYELKIS AVENDAÑO	4-810-1251

DOCENTE:

RAMÓN G. GONZÁLEZ TRIBALDOS

FECHA DE ENTREGA

/7/2026

1- ¿cómo puedo utilizar regex en gestión de errores?

R: Las expresiones regulares (Regex) son una herramienta utilizada para validar el formato de los datos que ingresan a un sistema, permitiendo gestionar errores de forma preventiva. En lugar de esperar a que un dato incorrecto provoque un fallo durante la ejecución del programa o al almacenarse en una base de datos. Si el dato es válido, el sistema continúa con el proceso; si no lo es, se genera un error controlado y se muestra un mensaje que indica al usuario cómo corregir la información.

En la gestión de errores, Regex es especialmente útil para validar datos como direcciones de correo electrónico, números de teléfono, contraseñas, nombres de usuario, fechas, códigos postales y otros campos que requieren un formato específico.

Regex actúa como un filtro: revisa cada dato ingresado y solo permite que continúen aquellos que cumplen el formato establecido, mientras que los datos incorrectos son rechazados mediante una gestión de errores controlada.

El uso de expresiones regulares también mejora la calidad de los datos almacenados, ya que impide que información incompleta o mal escrita llegue a la base de datos. Esto reduce la aparición de errores posteriores en el sistema, facilita el mantenimiento de la información y mejora la experiencia del usuario al proporcionar mensajes claros y específicos sobre el problema detectado. Además, al realizar la validación antes de procesar los datos, se disminuye el riesgo de fallos en otras partes de la aplicación.

El funcionamiento sucede en cuatro pasos:

1. El usuario ingresa un dato.
-Ejemplo: Jonatha@gmail.com
2. Se crea una expresión regular (Regex).
-Esta define el formato permitido.
3. El programa compara el dato con el patrón.
-Si el dato cumple el formato, el proceso continúa.
-Si no lo cumple, el sistema detecta el error.
4. Se gestiona el error.
-Se muestra un mensaje al usuario indicando qué debe corregir.

2- ¿Cuál es el formato de las expresiones regulares? (Estructura básica del patrón)

R: En PHP, las expresiones regulares están compuestas de tres elementos, delimitador, patrón y modificadores.

- Delimitador: Es el carácter que marca el inicio de una expresión regular, este puede ser cualquier carácter no alfanumérico, que no sea una barra invertida ni un espacio en blanco, los caracteres más usados son `/`, `#` y `~`.
- Patrón: La secuencia de caracteres que se quiere buscar.
- Modificadores: Según el sitio web códigoFacilito, los modificadores se utilizan para modificar la forma en que se evalúan las expresiones regulares. Se incluyen justo después del delimitador de la expresión regular y se pueden utilizar varios a la vez (Gallegos, 2019). Los modificadores más comunes son:
 - `i` (`PCRE_CASELESS`): Realiza una búsqueda sin distinguir entre mayúsculas y minúsculas
 - `m` (`PCRE_MULTILINE`): Realiza una búsqueda multilínea (los patrones que buscan una coincidencia al principio o al final de una cadena ahora coincidirán con el principio o el final de cada línea)
 - `u` (`PCRE_UTF8`): Permite la coincidencia correcta de patrones codificados en UTF-8

3- Ejemplo de implementación:

```

$mensaje = "";

if ($_SERVER["REQUEST_METHOD"] == "POST") {
    try {
        $nombre = trim($_POST["nombre"]);
        $correo = trim($_POST["correo"]);
        $telefono = trim($_POST["telefono"]);

        if ($nombre == "" || $correo == "" || $telefono == "") {
            throw new Exception("Debe llenar todos los campos.");
        }

        // Solo acepta letras y espacios.
        if (!preg_match("/^[A-Za-zÁÉÍÓÚáéíóúÑñ ]{3,40}$/", $nombre)) {
            throw new Exception("El nombre no cumple con el formato.");
        }

        // Formato sencillo de correo.
        if (!preg_match("/^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$/", $correo)) {
            throw new Exception("El correo no es valido.");
        }

        // Debe quedar como 0000-0000.
        if (!preg_match("/^[0-9]{4}-[0-9]{4}$/", $telefono)) {
            throw new Exception("El telefono debe tener el formato 0000-0000.");
        }

        $mensaje = "Datos validados correctamente.";
    } catch (Exception $e) {
        $mensaje = "Error: " . $e->getMessage();
    }
}
?>

```

Regex y manejo de errores

Registro de contacto

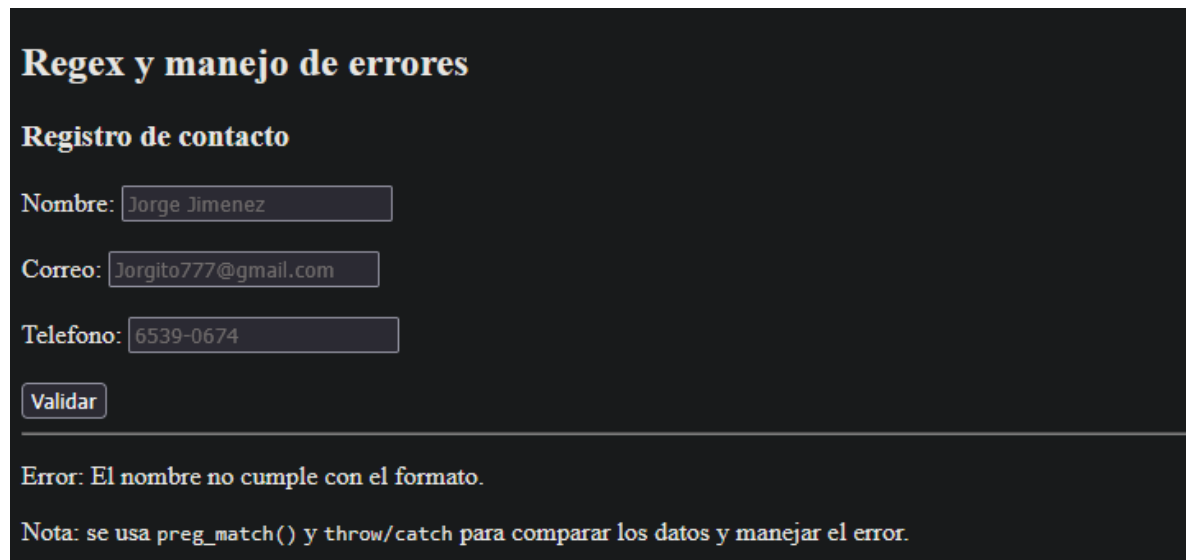
Nombre:

Correo:

Telefono:

Al presionar el botón de validar, se deberá mostrar el mensaje “Error: El nombre no cumple con el formato” porque, el campo de texto de “Nombre” tiene números adentro y al procesar el input a PHP, va a arrojar un error porque el código detectó números en los caracteres de la variable nombre, y utilizando expresiones

regulares, podemos filtrar números o caracteres especiales de ser aceptados en una cadena ingresada.



The screenshot shows a web form titled "Regex y manejo de errores" with a subtitle "Registro de contacto". It contains three input fields: "Nombre:" with the value "Jorge Jimenez", "Correo:" with the value "Jorgito777@gmail.com", and "Telefono:" with the value "6539-0674". Below these fields is a "Validar" button. A red error message is displayed below the button: "Error: El nombre no cumple con el formato." Below the error message is a note: "Nota: se usa preg_match() y throw/catch para comparar los datos y manejar el error."

Resultados de los datos ingresados.

Bibliografía

https://www.w3schools.com/php/php_regex.asp

<https://didesweb.com/tutoriales-php/expresiones-regulares-php/>

<https://www.php.net/manual/es/reference.pcre.pattern.modifiers.php>

https://codigofacilito.com/articles/articulo_16_10_2019_19_18_51